

5-DOF WITH GRIPPER ROBOTIC ARM USING ROBOT OPERATING SYSTEM (ROS)

A

REPORT

ON

**PROJECT OF
ELECTRONICS SYSTEM DESIGN LABORATORY
(3EL13)**

***Bachelor of Technology*
(Electronics Engineering)**

Submitted by:

**ROHIT VIJAY GUPTA
(21EL044)**

Under the guidance of:

Dr. Kaushika Patel



**Birla Vishvakarma Mahavidyalaya
Engineering Collage (An Autonomous Institution)**

Vallabh Vidyanagar - 388 120

Affiliated to Gujarat Technological University

CERTIFICATE

This is to certify that the Mini Project (2EL31) entitled “**5-DOF WITH GRIPPER ROBOT ARM USING ROBOT OPERATING SYSTEM (ROS)**” has been carried out by ***Rohit Gupta*** in the 5th semester under my guidance in Electronics Engineering, Birla Vishvakarma Mahavidyalaya, Vallabh Vidyanagar, during the academic year 2022- 23.

Guided by: **Dr. Kaushika Patel**

ACKNOWLEDGMENT

It is indeed pleasure and pride moment for us to undertake this Mini Project entitled **“5-DOF WITH GRIPPER ROBOT ARM USING ROBOT OPERATING SYSTEM (ROS)”**. We would like to thank **Dr. Kaushika Patel**, and **Dr. T.D.Pawar**, Head of Electronics department, Birla Vishvakarma Mahavidyalaya, Vallabh Vidyanagar, Anand to inspire us to undertake this project and also to support during this project.

Table of Contents

Abstract.....	6
Chapter 1	
1. Introduction.....	7
1.1 What is ROS?.....	8
1.2 What is ROS Noetic?.....	9
2. System Architecture.....	10
2.1 Hardware.....	10
2.2 ROS Nodes.....	10
2.3 ROS Topics.....	10
2.4 ROS Services and Actions.....	10
2.5 Integration with Higher-level Modules.....	10
2.6 Block diagram.....	11
3. Literature review.....	12
Chapter 2	
4. Hardware Components.....	13
4.1 SG90.....	13
4.2 MG996R.....	13
4.3 ARDUINO.....	14
4.4 3.7V Lithium-ion Battery.....	15
4.5 18650 Battery Slot.....	15
4.6 Jumper Wire.....	16
4.7 3D-Printed Robotic Arm.....	16
5. Circuit Diagram and RQT Graph.....	17
5.1 Circuit Diagram.....	17
5.2 What is RQT Graph?.....	18
5.3 RQT Graph.....	18
6. Software Components.....	20
6.1 URDF (Unified Robot Description Format).....	20
6.2 RViz (ROS Visualization).....	20
6.3 MoveIt (Motion Planning Framework).....	21
6.4 Linking URDF, RViz, and MoveIt.....	21
6.5 Workflow Example.....	22

Chapter 3

7. Launching Step.....	23
7.1 Without MoveIt Package.....	23
7.2 With MoveIt Package.....	24
7.3 Integrating with Hardware.....	25
8. Results and Observations.....	26
9. Conclusion.....	26
10. Future Enhancements.....	27
10.1 Integrate Camera with ROS.....	28
10.2 Object Detection and Localization.....	28
10.3 Real-time Training and Operation.....	28
11. References.....	29
Appendix-A.....	30
Appendix-B.....	32

Abstract

This project presents the development of a 5-Degree-of-Freedom (5-DOF) robotic arm with a gripper, utilizing the Robotic Operating System (ROS). The system is constructed with an Arduino connected to a laptop running Ubuntu, where ROS is installed. The CAD model is transformed into a Unified Robot Description Format (URDF) using Fusion. The integration of MoveIt plugin in ROS facilitates the arm's operation, and pick-and-place functionalities are implemented through Python code. The project demonstrates a comprehensive approach to robotic arm manipulation and control, showcase the coordination between hardware, software, and simulation within the ROS framework.

Chapter 1

1. Introduction

Robotics has revolutionized various industries by introducing automated systems that can perform complex tasks with precision and efficiency. Robotic arms, in particular, are versatile machines used in manufacturing, research, healthcare, and more. These arms can be equipped with different end-effectors such as grippers, suction cups, or specialized tools to perform a wide range of tasks.

In this report, we focus on a 5 Degrees of Freedom (5-DOF) robotic arm integrated with a gripper, controlled using the Robotic Operating System (ROS). ROS is an open-source framework that provides a collection of software tools and libraries to help developers build complex and robust robotic systems. It offers functionalities such as hardware abstraction, device drivers, messaging between processes, and package management, making it ideal for developing and controlling robotic arms.

The 5-DOF robotic arm refers to an articulated arm mechanism that can move in five different ways or axes. These typically include rotations at the base, shoulder, elbow, wrist pitch, and wrist roll. The addition of a gripper allows the arm to grasp and manipulate objects, enhancing its versatility and practical applications.

The primary objectives of this report are as follows:

- 1. System Overview:** Providing an overview of the hardware components comprising the 5-DOF robotic arm and gripper system, including actuators, sensors, and the gripper mechanism.
- 2. ROS Integration:** Exploring the integration of the robotic arm with ROS, including the setup of ROS nodes, communication protocols, and message passing for controlling the arm's movements and gripper operations.
- 3. Kinematics and Motion Planning:** Discussing the kinematic model of the robotic arm and implementing motion planning algorithms within ROS to enable efficient and accurate movement trajectories.
- 4. Manipulation and Applications:** Demonstrating practical applications of the robotic arm, such as pick-and-place tasks, object manipulation, and potential use cases in industrial automation .

1.1 WHAT IS ROS?

ROS, short for Robot Operating System, is an open-source framework specifically designed to help developers create robotic software. Despite its name, ROS is not a full-fledged operating system like Windows or Linux; instead, it provides a set of software libraries, tools, and conventions aimed at simplifying the process of creating complex and robust robot behavior across a wide variety of platforms.

Key features of ROS include:

- 1. Message Passing:** ROS facilitates communication between different parts of a robot system using a publish-subscribe messaging model. Nodes (independent processes) can send and receive messages (data) over topics, allowing modular and scalable architecture.
- 2. Package Management:** ROS uses a package-based architecture to organize and distribute code. Packages encapsulate libraries, executables, datasets, configuration files, and more. This modular approach encourages code reuse and collaboration.
- 3. Hardware Abstraction:** ROS provides standardized interfaces (such as drivers) for interacting with robot hardware, enabling developers to write robot-agnostic code. This abstraction simplifies development and promotes interoperability.
- 4. Tools and Visualization:** ROS comes with a suite of command-line tools and graphical user interfaces (GUIs) for visualization, debugging, and system introspection. Tools like `rviz` for 3D visualization and `rqt` for GUI-based introspection are widely used.
- 5. Community and Ecosystem:** ROS has a large and active community that contributes packages, tutorials, and expertise. This ecosystem supports developers in rapidly prototyping, testing, and deploying robotic systems.

Some of the notable versions of ROS (Robot Operating System) include:

- ROS Kinetic
- ROS Melodic
- ROS Noetic
- ROS 2 Dashing
- ROS 2 Foxy

In this project we have used **ros noetic**.

1.2 ROS Noetic

ROS Noetic is a specific version of ROS, named after the North American box turtle . It was released in May 2020 and is the latest Long-Term Support (LTS) version of ROS as of my last update. LTS versions of ROS receive support for a longer period, making them suitable for production deployments and long-term projects.

Key characteristics of ROS Noetic include:

- 1. Python 3 Compatibility:** ROS Noetic is designed to work with Python 3, unlike previous versions that primarily supported Python 2. This transition aligns with broader industry trends and improves compatibility with modern software libraries.
- 2. Ubuntu 20.04 Support:** ROS Noetic officially supports Ubuntu 20.04 (Focal Fossa) as its primary platform. This compatibility ensures ROS Noetic can take advantage of the latest system libraries and improvements.
- 3. Package Updates:** ROS Noetic introduces updates and improvements to existing packages, including better support for newer hardware and libraries.
- 4. Continued Development and Community:** ROS Noetic benefits from ongoing development efforts by the ROS community, ensuring compatibility with the latest technologies and addressing user feedback.

2. SYSTEM ARCHITECTURE

The system architecture of a 5-degree-of-freedom (5-DOF) robotic arm with a gripper using the Robotic Operating System (ROS) typically involves several key components and nodes working together within the ROS framework. Below is an overview of the system architecture:

2.1. Hardware:

- **Robotic Arm:** The physical structure of the robotic arm consists of joints and links that allow movement along five degrees of freedom (e.g., base rotation, shoulder, elbow, wrist pitch, wrist roll).
- **Gripper:** A robotic gripper or end-effector attached to the arm for grasping and manipulating objects.

2.2. ROS Nodes:

- **Joint State Publisher:** This node is responsible for publishing the current state (position) of each joint of the robotic arm. It collects joint angle data from sensors or control systems and publishes this information to ROS topics.
- **Inverse Kinematics Solver:** This node computes the joint angles required to achieve a desired end-effector pose (position and orientation) based on inverse kinematics calculations. It takes input from higher-level controllers or planning modules.
- **Trajectory Planner:** In some setups, a trajectory planner node may be used to generate smooth trajectories for the robotic arm's movement, ensuring optimal and collision-free paths.
- **Gripper Controller:** A node dedicated to controlling the gripper's state (open/close) based on commands received from higher-level modules or user inputs.
- **Sensor Interfaces:** Nodes interfacing with sensors (e.g., encoders, cameras) to provide feedback for state estimation, collision avoidance, or object detection tasks.

2.3. ROS Topics:

- **Joint State Topics:** These topics carry the current joint state information published by the Joint State Publisher node. Other nodes subscribe to these topics to obtain real-time joint position data.
- **Desired Pose Topics:** Topics used to specify the desired pose (position and orientation) of the end-effector. This information is typically consumed by the Inverse Kinematics Solver or Trajectory Planner.
- **Gripper Control Topics:** Topics for sending commands to control the gripper's state (open/close). The Gripper Controller node subscribes to these topics to actuate the gripper accordingly.

2.4. ROS Services and Actions:

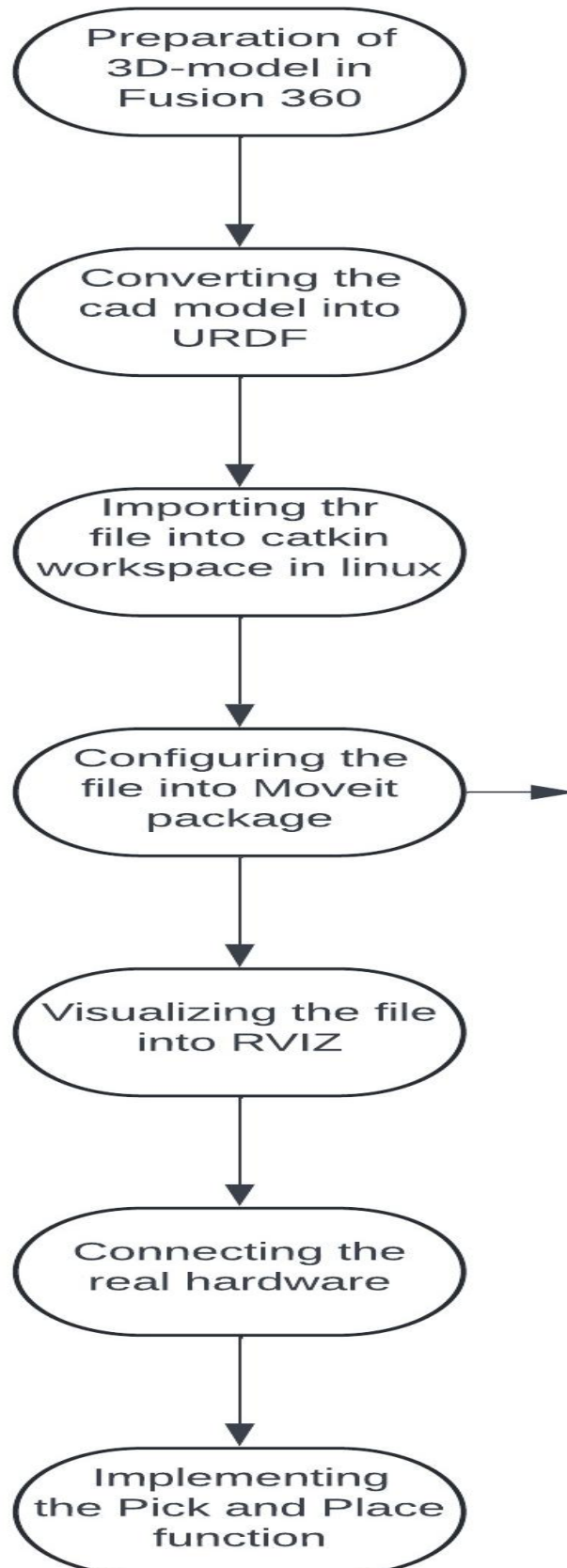
- **Inverse Kinematics Service:** A service that accepts a pose request (target end-effector position and orientation) and computes the corresponding joint angles using inverse kinematics calculations.
- **Trajectory Execution Action:** An action server used for executing planned trajectories. This allows for asynchronous trajectory execution and monitoring.

2.5. Integration with Higher-level Modules:

- The robotic arm system architecture in ROS can be integrated with higher-level modules such as motion planners, task planners, perception systems, and human-machine interfaces to enable complex functionalities like autonomous operation, task execution, and interaction with the environment.

2.6 BLOCK DIAGRAM

block diagram for a 5-DOF robotic arm with a gripper using the Robot Operating System (ROS),



3. Literature review

1) “ROS Based Autonomous Mobile Manipulator Robot.” by Sufiyan Ali, et al

In this research paper, get to know how to link to configure your real hardware with rviz model and get to know about slam, camera and how to use that in navigating the robot in target location and how to pick up and place the target body through camera.

2) “Economic 6-DOF Robotic Manipulator Hardware Design for Research and Education” by Vyas, Dhaval R., et al.

In this research paper, we get to learn about how to configure the CAD description model i.e. urdf in moveit package and how to connect the hardware to ROS package.

3) “Integrating Soft Robotics with ROS: A Hybrid Pick and Place Arm*” by Ross M. McKenzie

In this paper, learned how to write the pick and place code which can be implemented on both rviz and real hardware.

Chapter 2

4. Hardware Components

4.1 Sg-90 servo motor(3 nos)

The SG90 servo is a popular micro-sized servo motor that is widely used in hobbyist projects, robotics, and remote-controlled models. Here are some key details and specifications about the SG90 servo:

1. Size and Weight:
 - Size: 23 mm x 12.2 mm x 29 mm (approximate)
 - Weight: 9 grams (approximately)
2. Operating Voltage:
 - Typically operates at 4.8V to 6V DC.
3. Torque:
 - At 4.8V: Approximately 1.8 kg-cm
 - At 6V: Approximately 2.2 kg-cm
4. Speed:
 - At 4.8V: Approximately 0.12 seconds/60 degrees.
 - At 6V: Approximately 0.10 seconds/60 degrees.



5. Control Interface:
 - The SG90 servo motor typically uses a standard 3-wire (power, ground, control) connection.
 - Control signal is a PWM (Pulse Width Modulation) signal.
 - Operating angle is generally 180 degrees.
6. Compatibility:
 - Compatible with most microcontroller platforms like Arduino, Raspberry Pi, PIC, etc.
 - Requires a separate power supply (4.8V to 6V) to operate.
7. Applications:
 - Used in model airplanes, helicopters, cars, and boats for controlling various parts like control surfaces, steering mechanisms, and more.
 - Commonly used in robotic projects for actuating joints and other moving parts.
 - Suitable for small-scale automation and control applications.

4.2 MG996r(3 nos)

The MG996R is another popular servo motor used in robotics, RC vehicles, and other projects requiring precise and powerful motion control. Here's a detailed overview of the MG996R servo:

1. Size and Weight:
 - Size: 40.7 mm x 19.7 mm x 42.9 mm
 - Weight: 55 grams
2. Operating Voltage:
 - Typically operates at 4.8V to 7.2V DC.
3. Torque:
 - At 4.8V: Approximately 9.4 kg-cm
 - At 6V: Approximately 11 kg-cm
 - At 7.2V: Approximately 13 kg-cm



4. Speed:

- At 4.8V: Approximately 0.17 seconds/60 degrees
- At 6V: Approximately 0.14 seconds/60 degrees
- At 7.2V: Approximately 0.12 seconds/60 degrees

5. Control Interface:

- Uses a standard 3-wire (power, ground, control) connection.
- Control signal is a PWM (Pulse Width Modulation) signal.
- Operating angle is generally 180 degrees.

6. Compatibility:

- Compatible with most microcontroller platforms like Arduino, Raspberry Pi, PIC, etc.
- Requires a separate power supply (4.8V to 7.2V) to operate.

7. Applications:

- Used in larger robotic projects, RC cars, boats, and other applications requiring higher torque and precision.
- Suitable for projects where more force and reliability are needed compared to smaller servos like the SG90.

4.3 ARDUINO

Arduino is an open-source electronics platform based on easy-to-use hardware and software. It consists of both physical programmable circuit boards (microcontroller boards) and a development environment (IDE) for writing and uploading code to the board. The platform was designed to make electronics and programming accessible to artists, designers, hobbyists, and anyone interested in creating interactive projects.

Key Components of Arduino:

1. Arduino Boards:

- The Arduino boards are small microcontroller-based computers that can be programmed to control electronic devices. They come in various models and sizes, each with different capabilities and features.
- Common Arduino boards include Arduino Uno, Arduino Mega, Arduino Nano, Arduino Due, etc.

2. Microcontroller:

- At the heart of every Arduino board is a microcontroller (typically from the Atmel AVR family or ARM processors). This is what runs the code uploaded from the Arduino IDE.

3. Arduino IDE (Integrated Development Environment):

- The Arduino IDE is a software application used to write, compile, and upload code (sketches) to Arduino boards. It provides a simple interface for writing programs (sketches) using the Arduino programming language, which is based on C/C++.

4. Arduino Programming Language:

- Arduino uses its own programming language, which is very similar to C/C++. It abstracts away many of the complexities of traditional programming languages, making it easier for beginners to get started with electronics and programming.

5. Library Support:

- Arduino comes with a vast collection of libraries that simplify the process of interfacing with various hardware components such as sensors, motors, displays, communication modules, etc.



4.4 3.7v lithium ion battery (2 nos)

A 3.7V lithium-ion battery (Li-ion battery) is a type of rechargeable battery commonly used in various electronic devices ranging from smartphones and laptops to drones and electric vehicles. Here's a detailed overview of the 3.7V lithium-ion battery:

Key Characteristics:

1. Voltage:

- A fully charged lithium-ion battery typically provides a voltage of 4.2V. As it discharges, the voltage decreases down to around 3.0V when considered empty. The average voltage during discharge is approximately 3.7V, hence the name "3.7V lithium-ion battery."

3. Capacity:

- The capacity of a lithium-ion battery is usually measured in milliamper-hours (mAh) or ampere-hours (Ah). Common capacities range from a few hundred milliamper-hours for small applications (e.g., smartwatches) to several thousand milliamper-hours for larger devices (e.g., laptops).
- For example, a typical 3.7V lithium-ion battery might have a capacity of 2000mAh, which means it can supply 2 amps of current for 1 hour before being fully discharged.

4. Rechargeability:

- Lithium-ion batteries are rechargeable and can be recharged hundreds to thousands of times, depending on the specific battery chemistry and usage conditions.

5. Safety:

- Proper handling and charging of lithium-ion batteries are essential for safety. They should be charged using dedicated chargers designed for lithium-ion batteries to prevent overcharging and over-discharging, which can lead to safety hazards like fire or explosion.

6. Applications:

- Lithium-ion batteries are widely used in portable electronic devices such as smartphones, tablets, digital cameras, and power banks due to their high energy density and relatively light weight.
- They are also used in larger applications like electric vehicles (EVs) and energy storage systems (ESS) for renewable energy sources due to their ability to store large amounts of energy.



4.5 18650 Battery slot

An "18650 battery slot" refers to a specific type of battery holder or receptacle designed to accommodate an 18650 lithium-ion battery.

The 18650 battery is a popular cylindrical lithium-ion cell format, named for its dimensions: 18mm in diameter and 65mm in length.

These batteries are widely used in various applications due to their high energy density, rechargeability, and standardized size.



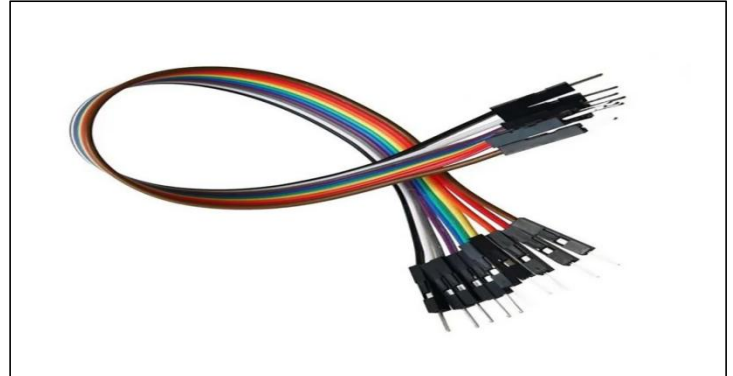
4.6 Jumper Wire

Jumper wires are essential components in electronics and prototyping that are used to create temporary electrical connections between different points on a breadboard, circuit board, or electronic components. They are typically insulated wires with connectors at both ends, allowing for easy and quick connections without the need for soldering.

Types of Jumper Wires:

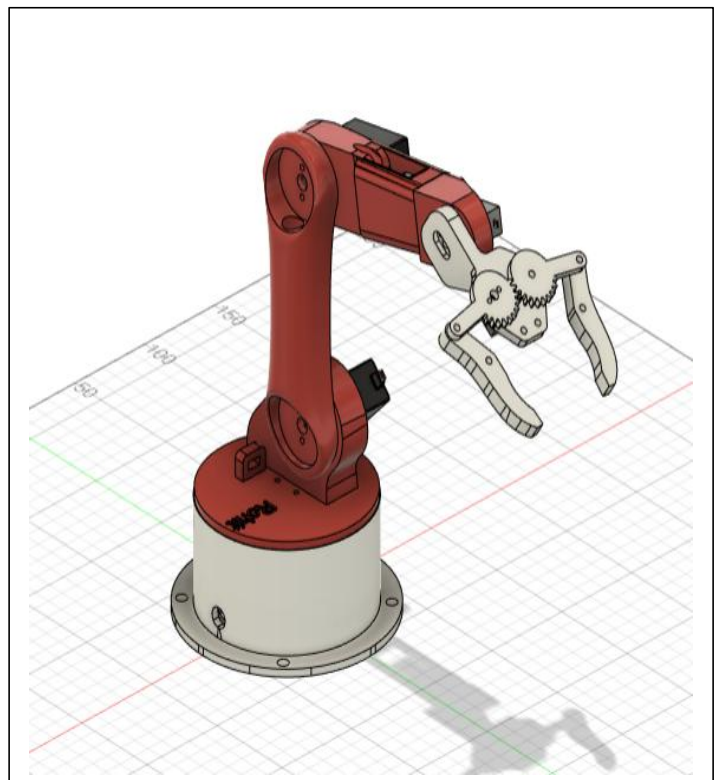
Jumper wires often come with various types of connectors at each end, including:

1. Male-to-Male (pin-to-pin) connectors: Both ends have pins, suitable for connecting between female headers or directly into breadboard holes.
2. Female-to-Female (socket-to-socket) connectors: Both ends have sockets, used for extending connections between pins or components.
3. Male-to-Female (pin-to-socket) connectors: One end has pins and the other end has a socket, ideal for connecting components to a breadboard or extending connections.



4.7 3-D Printed Robotic Arm

A 3D printed robotic arm combines the versatility of additive manufacturing with the functionality of robotic automation. Using 3D printing technology, intricate and custom-designed parts can be created with precision and efficiency. The robotic arm's joints and links are typically designed and optimized for strength, flexibility, and weight reduction. This approach allows for rapid prototyping and customization, making it ideal for research, education, and hobbyist projects. With 3D printing, complex robotic arm designs can be realized quickly and cost-effectively, empowering innovation in robotics and automation.

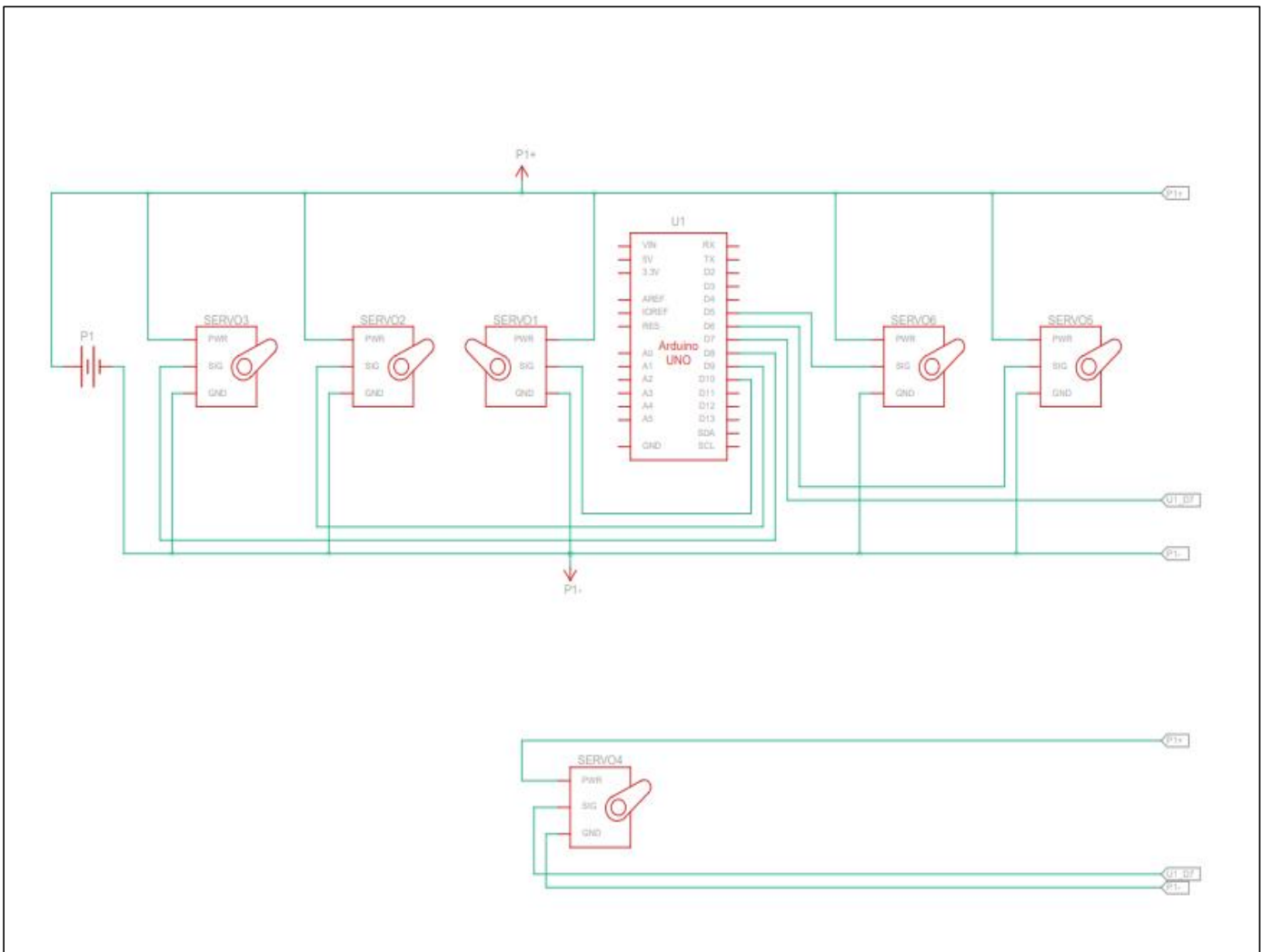


5. Circuit Diagram and RQT graph

To provide a comprehensive overview of the system architecture for a 5-DOF robotic arm using ROS, we'll discuss two key components: the circuit diagram representing the hardware connections and the RQT graph illustrating the ROS nodes and topics.

5.1 Circuit Diagram

The circuit diagram for a 5-DOF robotic arm with a gripper involves illustrating the electrical connections between the various components of the robotic system.



5.2 WHAT IS RQT GRAPH

The RQT Graph (ROS Graph) is a powerful visualization tool used in ROS (Robot Operating System) to display the network of nodes and topics within a ROS system. It provides a graphical representation of how different components of a ROS-based application communicate with each other in real-time. Understanding the RQT Graph is essential for debugging, monitoring, and gaining insights into the behavior of ROS systems. Here's a detailed explanation of the RQT Graph components and how to interpret them:

Components of the RQT Graph:

1. Nodes:

- **Definition:** Nodes are individual software processes in ROS that perform specific tasks, such as controlling hardware, processing sensor data, or implementing algorithms.
- **Representation:** In the RQT Graph, nodes are depicted as rectangular boxes labeled with their respective names (e.g., `roscore`, `joint_state_publisher`, `robot_controller`).
- **Purpose:** Nodes communicate with each other by publishing or subscribing to topics, providing a modular and distributed architecture in ROS.

2. Topics:

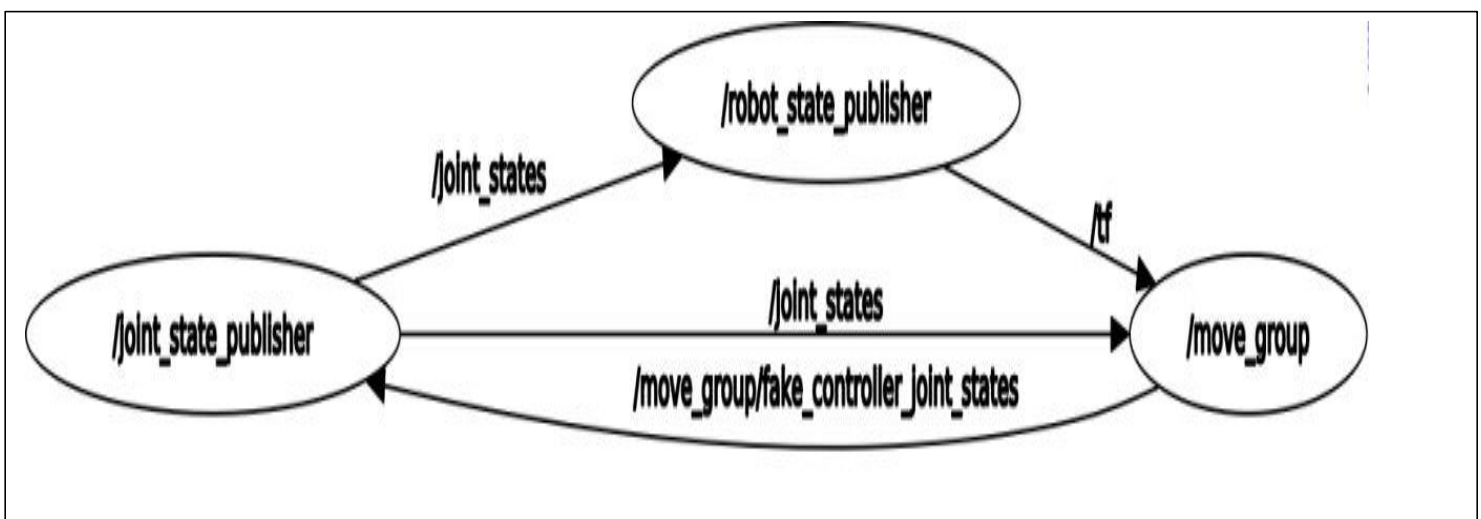
- **Definition:** Topics are named communication channels in ROS through which nodes exchange messages. They can be thought of as data buses that enable communication between nodes.
- **Representation:** Topics are depicted as ellipses labeled with their names (e.g., `/joint_states`).
- **Purpose:** Nodes publish messages to topics to share information, and other nodes subscribe to topics to receive and process these messages.

3. Connections (Edges):

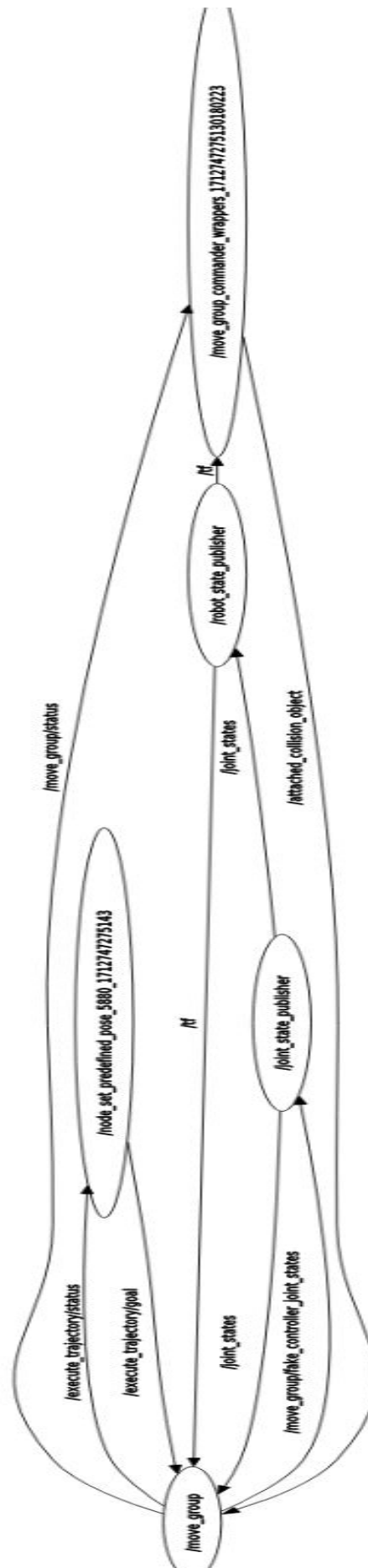
- **Definition:** Connections represent the flow of data between nodes and topics in the ROS system. They indicate which nodes are publishing messages to which topics and which nodes are subscribing to which topics.
- **Representation:** Connections are represented by arrows pointing from nodes to topics (publishing) or from topics to nodes (subscribing) in the RQT Graph.
- **Purpose:** Connections visualize the data flow and dependencies within the ROS system, helping to identify communication patterns and troubleshoot issues related to message passing.

6.3 RQT GRAPH

1. BASIC (without pickup and place script)



2. With pickup and place script



6. Software Components

The software components are URDF (Unified Robot Description Format), RViz (ROS Visualization), and MoveIt (Motion Planning Framework)—are essential tools in ROS (Robot Operating System) for modeling, visualizing, and planning motion for robotic systems. Let's discuss how these components are linked together to create a comprehensive robotics development environment.

6.1 URDF (Unified Robot Description Format):

URDF is an XML-based file format used to describe the physical properties, kinematics, and visual appearance of a robot model.

- 1. Robot Modeling:** URDF files define the robot's structure, including links (physical components) and joints (connections between links).
- 2. Visualization:** URDF models can be rendered in RViz for visual inspection and debugging.
- 3. Simulation:** URDF models are used in robot simulation environments like Gazebo to simulate robot behavior.

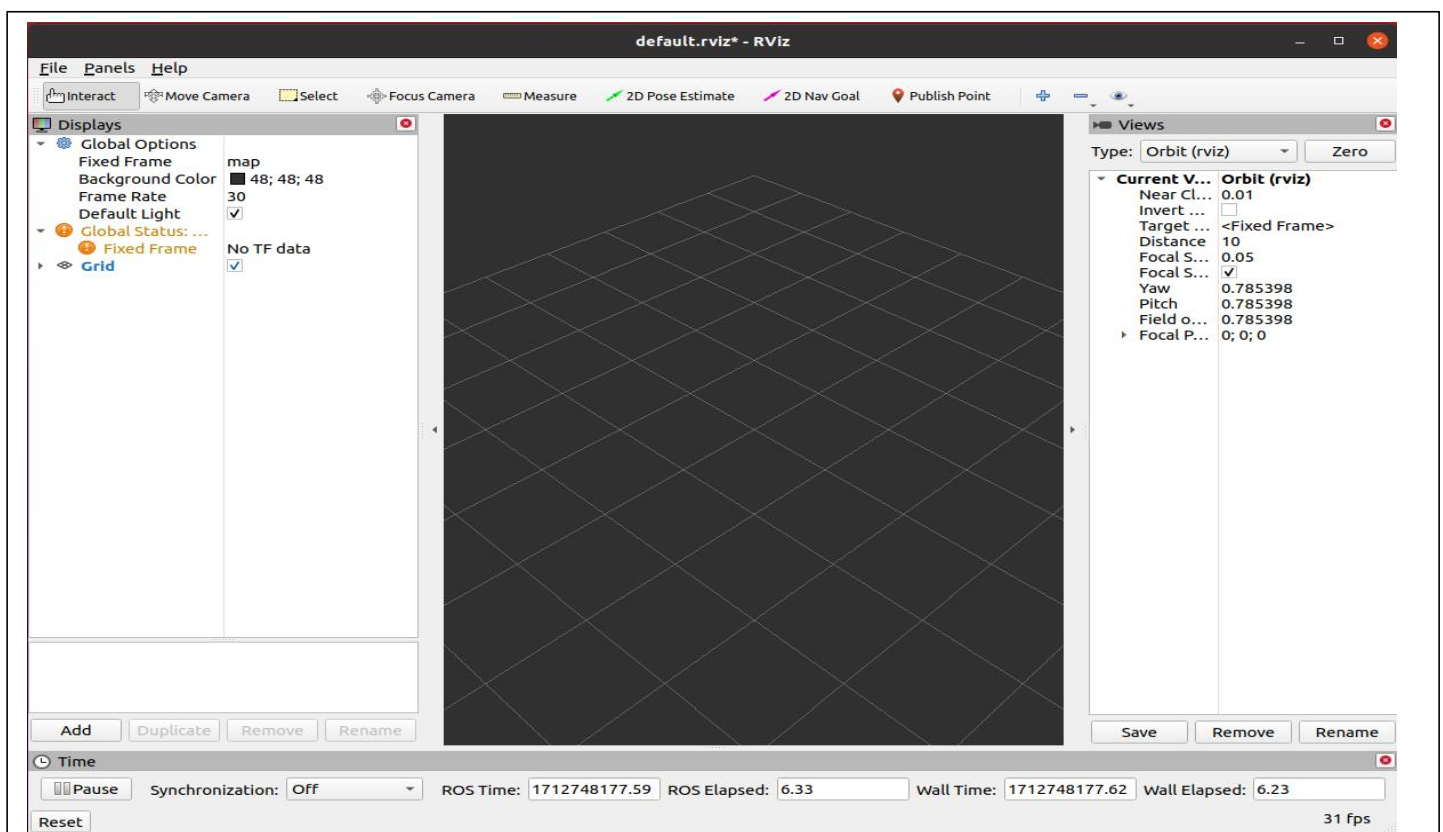
This is generate through fusion 360 cad desiging tool where you have to add script name fusiontourdf

6.2 RViz (ROS Visualization):

RViz is a powerful 3D visualization tool in ROS that allows users to visualize sensor data, robot models, and other ROS messages in real-time.

- 1. Robot Visualization:** RViz can display URDF models of robots, showing their joint configurations, sensor data, and trajectories.
- 2. Debugging and Analysis:** RViz helps developers debug and analyze robot behavior by visualizing sensor outputs and motion planning results.
- 3. User Interface:** RViz provides an interactive graphical interface for monitoring and interacting with ROS topics and messages.

Open through command in terminal - **rviz**

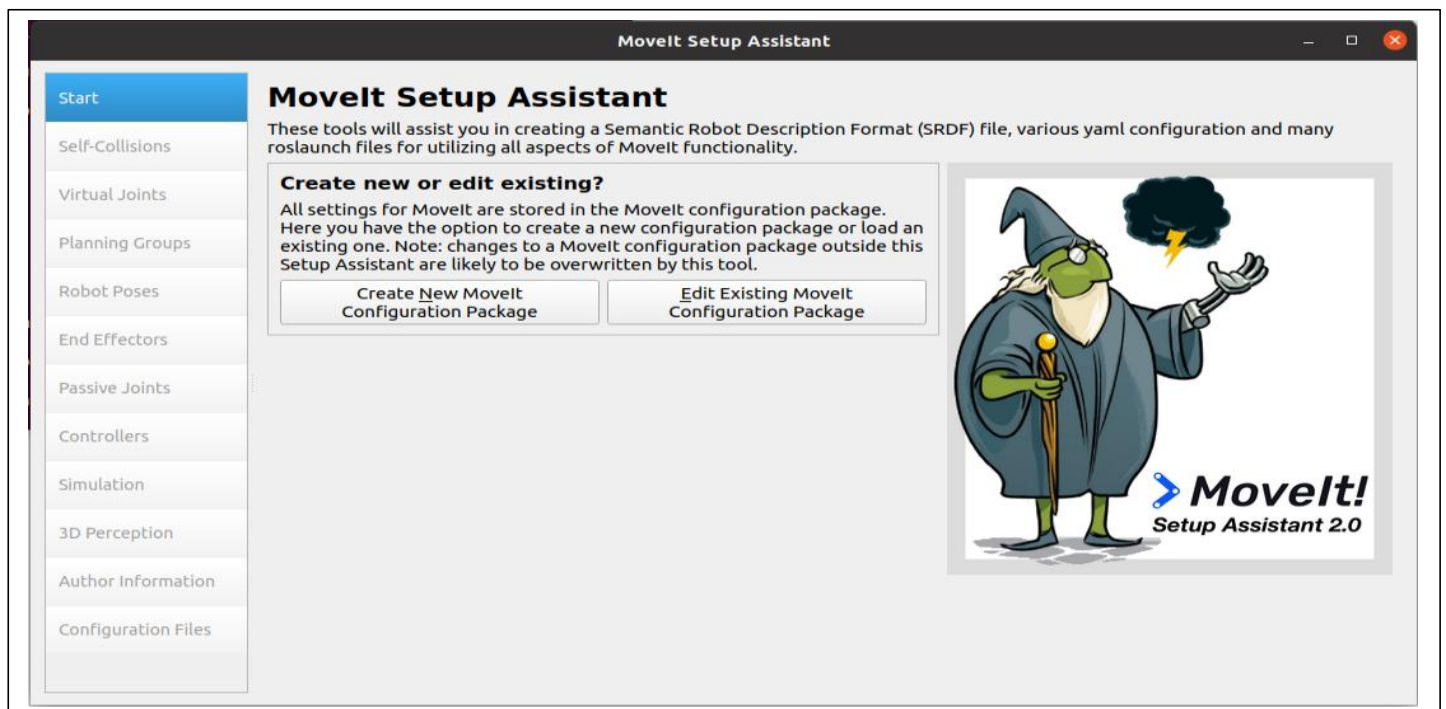


6.3 MoveIt (Motion Planning Framework):

MoveIt is a ROS library and framework specifically designed for motion planning, kinematics, and manipulation of robotic arms.

1. **Motion Planning:** MoveIt uses the URDF model to perform motion planning tasks, such as path planning and collision avoidance.
2. **Grasping and Manipulation:** MoveIt includes tools for grasping objects and performing manipulation tasks using robot arms.
3. **Integration with RViz:** MoveIt seamlessly integrates with RViz to visualize planned trajectories and interactively modify motion plans.

Open through command in terminal - `roslaunch moveit_setup_assistant setup_assistant.launch`



6.4 Linking URDF, RViz, and MoveIt:

1. URDF Modeling:

- Create or modify URDF files to accurately represent the physical structure and kinematics of the robot.

2. RViz Visualizatio:

- Load the URDF model into RViz to visualize the robot's appearance, joint states, and sensor data.
- Use RViz's interactive tools to control the robot's joints, view sensor outputs, and debug motion planning algorithms.

3. MoveIt Integration:

- Configure MoveIt to use the URDF model for motion planning and collision checking.
- Plan and execute robot motions using MoveIt's APIs, which leverage the URDF model's kinematic information.
- Visualize planned trajectories and robot configurations in RViz, allowing for interactive motion planning and manipulation tasks.

6.5 Workflow Example:

1. URDF Creation:

- Design and define the robot's structure and kinematics in a URDF file.

2. RViz Visualization:

- Load the URDF model into RViz to visualize the robot's appearance and joint configurations.

3. MoveIt Configuration:

- Configure MoveIt to use the URDF model for motion planning and kinematic calculations.

4. Motion Planning:

- Use MoveIt's APIs to plan and execute motions (e.g., reaching a target pose, grasping an object) based on the URDF model.

5. Visualization and Interaction:

- Visualize planned trajectories and robot states in RViz, enabling interactive debugging and refinement of motion plans.

Chapter 3

7. launching step

To effectively use URDF, RViz, and MoveIt together for robot modeling, visualization, and motion planning in ROS, follow these step-by-step guidelines:

Assuming that you have install ubuntu 20.04 lts version and install ROS noetic in it

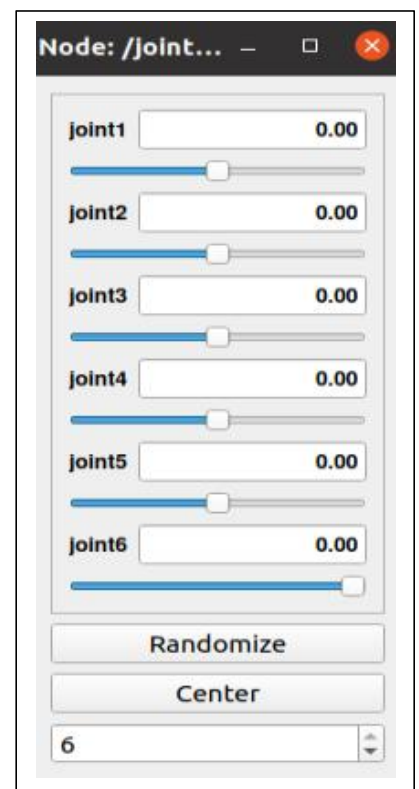
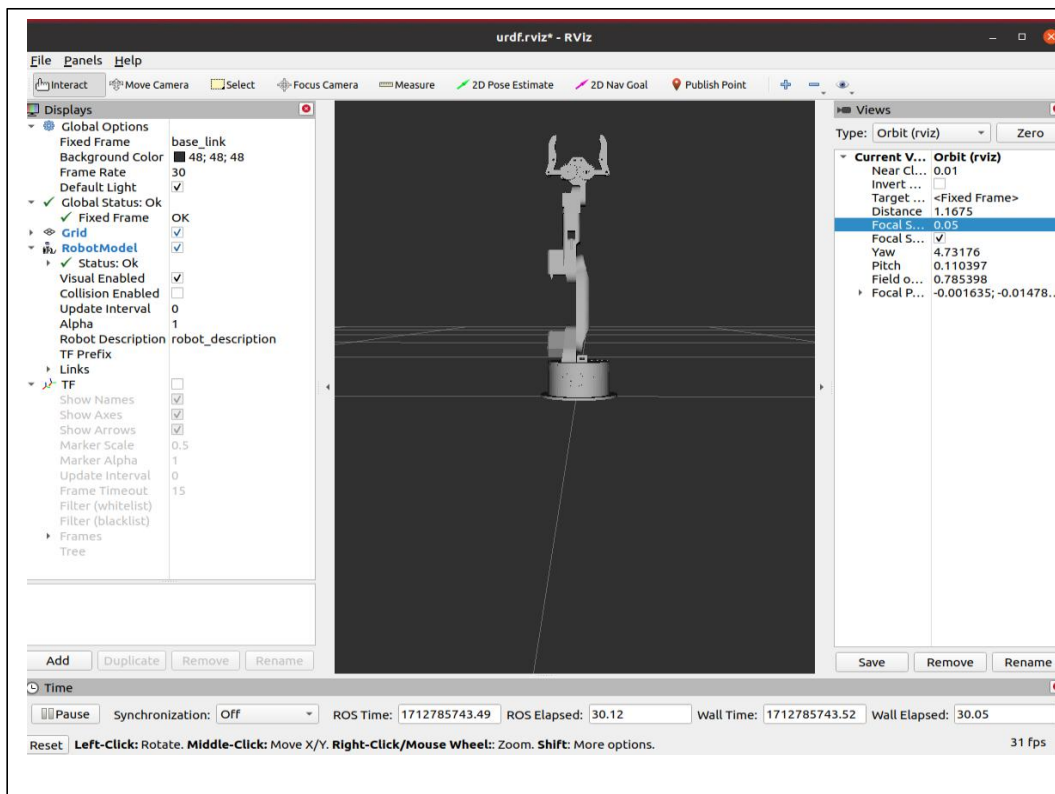
7.1 Without Moveit package

- Open terminal 1
- Run command :- **roscore**
- Open terminal 2
- Run command :- **mkdir robotic_arm && cd robotic_arm**
- Run command :- **mkdir src**
- Run command : **cd src**
- Run command :- **git clone <https://github.com/Rohitvgupta15/5-DOF-ROBOTIC-ARM-WITH-GRIPPER.git>**
- Run command :- **catkin_make**
- Run command :- **cd ..**
- Run command :- **roslaunch arm_5dof_description display.launch**

To run pick_and_place script

- Oprn terminal 3
- Run command :- **cd robotic_src**
- Run command :- **roslaunch moveit_pkg pick_and_place.py**

Here , we can observe our model in rviz and can control each joint through controller.

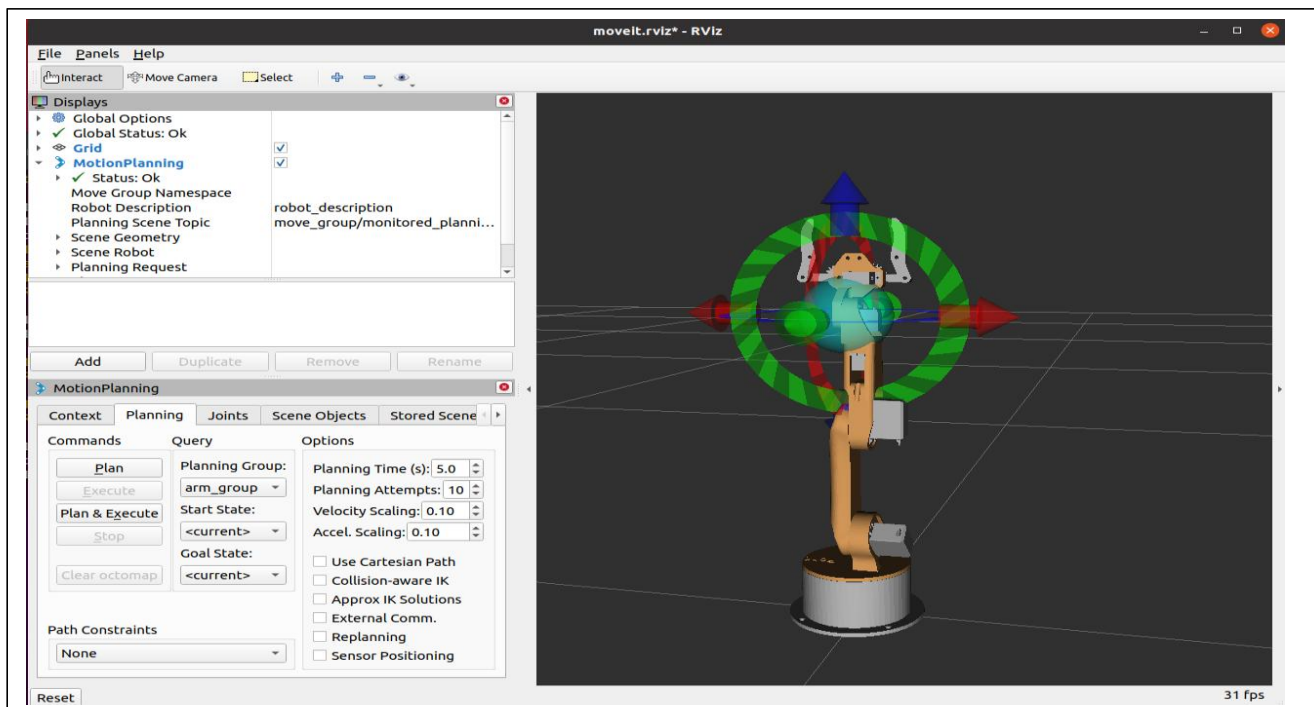


7.2 with moveit package

- Open terminal 1
- Run command :- **roscore**
- Open terminal 2
- Run command :- **cd robotic_arm**
- Run command :- **roslaunch moveit_pkg demo.launch**

To run pick_and_place script

- Oprn terminal 3
- Run command :- **cd robotic_src**
- Run command :- **roslaunch moveit_pkg pick_and_place.py**



```

rohit@rohit:~/robotic_arm/src/moveit_pkg/launch/demo.launch http://localhost:11311

rohit@rohit:~/robotic_arm$ source devel/setup.bash
rohit@rohit:~/robotic_arm$ roslaunch moveit_pkg pick_and_place.py
[ INFO] [1712785868.728011437]: Loading robot model 'arm_5dof'...
[ INFO] [1712785868.728965121]: No root/virtual joint specified in SRDF. Assuming fixed joint
[ WARN] [1712785868.930478573]: Could not identify parent group for end-effector 'end_effector'
[ WARN] [1712785868.964655119]: The root link base_link has an inertia specified in the URDF, but KDL
or URDF.
[ INFO] [1712785870.069186867]: Ready to take commands for planning group arm_group.
[ INFO] [1712785870.301620]: Planning Group: base_link
[ INFO] [1712785870.303338]: End Effector Link: gripper_base_1
[ INFO] [1712785870.304985]: Group Names: ['arm_group', 'hand']
[ INFO] [1712785870.306581]: >>> MyRobot initialization is done.
[ INFO] [1712785870.657939948]: Ready to take commands for planning group hand.
[ INFO] [1712785870.662461]: Planning Group: base_link
[ INFO] [1712785870.664762]: End Effector Link: gear1_1
[ INFO] [1712785870.667026]: Group Names: ['arm_group', 'hand']
[ INFO] [1712785870.669470]: >>> MyRobot initialization is done.
[ INFO] [1712785870.670992]: Going to Pose: home
[ INFO] [1712785873.417707]: Now at Pose: home
[ INFO] [1712785873.419802]: Going to Pose: hand_open
[ INFO] [1712785874.016423]: Now at Pose: hand_open
[ INFO] [1712785875.020232]: Going to Pose: pick_1
[ INFO] [1712785878.616407]: Now at Pose: pick_1
[ INFO] [1712785879.619478]: Going to Pose: hand_closed
[ INFO] [1712785883.019190]: Now at Pose: hand_closed
[ INFO] [1712785884.021773]: Going to Pose: pick_2
[ INFO] [1712785887.417211]: Now at Pose: pick_2
[ INFO] [1712785888.421501]: Going to Pose: pick_3

```

7.3 Integrating with Hardware

Command to be run on terminal

- open terminal 4
- Run command :- `roslaunch rosserial_python serial_node.py /dev/ttyUSB0 _baud:=115200`

```
rohit@rohit: ~/robotic
/home/rohit/robotic/src/arduino_rob... x rohit@rohit: ~/robotic x
rohit@rohit:~/robotic$ roslaunch rosserial_python serial_node.py /dev/ttyUSB0 _baud:=115200
[INFO] [1712834239.138185]: ROS Serial Python Node
[INFO] [1712834239.151391]: Connecting to /dev/ttyUSB0 at 115200 baud
[INFO] [1712834241.409787]: Requesting topics...
[INFO] [1712834241.522191]: Note: subscribe buffer size is 280 bytes
[INFO] [1712834241.526976]: Setup subscriber on joint_states [sensor_msgs/JointState]
```



8. Result and Observation

To observe the results and gather insights from using URDF, RViz, and MoveIt together in ROS for robot modeling, visualization, and motion planning, you can perform specific tasks and analyze the outcomes. Below are some typical results and observations you might encounter when working with these tools:

8.1 Results:

1. URDF Model Visualization in RViz:

- **Result:** The robot's URDF model is successfully visualized in RViz, showing the robot's structure, joint configurations, and visual appearance.
- **Observation:** You can observe how the robot looks in the virtual environment and verify if the URDF accurately represents the physical robot.

2. Interactive Motion Planning with MoveIt:

- **Result:** MoveIt successfully plans trajectories for the robot to reach desired poses or perform manipulation tasks.
- **Observation:** You can observe the planned trajectories in RViz, visualize collision-free paths, and verify if the robot can execute the planned motions effectively.

3. Real-time Robot State Visualization:

- **Result:** RViz displays real-time updates of the robot's joint states, sensor data, and planned trajectories.
- **Observation:** You can monitor how the robot's state changes during motion planning and execution, helping in debugging and validation.

4. Collision Detection and Avoidance :

- **Result:** MoveIt's collision detection capabilities prevent the robot from colliding with obstacles during motion planning.
- **Observation:** Use RViz to visualize collision objects and observe how MoveIt adjusts trajectories to avoid collisions, ensuring safety in robot operations.

5. Performance and Efficiency:

- **Result:** Evaluate the performance of motion planning algorithms in terms of speed, accuracy, and resource utilization.
- **Observation:** Measure the time taken for trajectory planning and execution, assess the efficiency of kinematic solvers, and optimize parameters based on observations.

9. Conclusion:

By observing the results and analyzing the outcomes of using URDF, RViz, and MoveIt in ROS, you can gain valuable insights into the performance, functionality, and usability of your robotic system. Use these observations to iterate on your designs, optimize algorithms, and enhance the overall effectiveness of your ROS-based robotic applications. The combination of modeling, visualization, and motion planning tools in ROS enables efficient development and deployment of complex robotic systems, advancing capabilities in robotics research, development, and automation.

10. Future Enhancements

To enhance your robotic system with the capability to detect specific objects using OpenCV, pick them up, and place them at predetermined locations, while also enabling real-time training for robot operations, you can follow a structured approach leveraging ROS, OpenCV, and machine learning techniques. Below are the steps and considerations for implementing these enhancements:

10.1. Integrate Camera with ROS

1.1 Camera Setup:

- Connect a compatible camera (e.g., USB camera, RGB-D sensor) to your ROS system.
- Use ROS camera drivers (e.g., `'usb_cam'`, `'openni_camera'`) to stream camera images as ROS topics.

1.2 Image Processing with OpenCV:

- Develop ROS nodes using OpenCV to subscribe to the camera image topic.
- Implement image processing algorithms (e.g., color detection, object segmentation) in OpenCV to identify specific objects of interest within the camera images.

10.2. Object Detection and Localization

2.1 Object Detection:

- Use OpenCV to detect and localize specific objects based on predefined characteristics (e.g., color, shape, texture).
- Implement robust object detection algorithms (e.g., Haar cascades, HOG features, deep learning models) depending on the complexity of the objects.

2.2 ROS Integration:

- Publish the detected object's pose (position and orientation) as ROS messages, enabling other nodes to access this information for further processing.

10.3. Real-time Training and Operation

3.1 Interactive Learning Interface:

- Implement a user interface or interactive tool using ROS and a GUI library (e.g., Qt) to enable real-time training and operation.
- Allow users to provide feedback or corrections during robot operations to refine the learning process.

3.2 Machine Learning Integration:

- Integrate machine learning techniques (e.g., reinforcement learning, imitation learning) to train the robot in real-time based on user interactions and feedback.
- Use ROS interfaces to communicate between machine learning models and robotic control nodes for seamless integration.

11. References

1. <https://automaticaddison.com/how-to-set-up-and-control-a-robotic-arm-using-moveit-and-ros/>
2. <https://vinaylanka.medium.com/control-any-robotic-arm-with-ros-b10a3115306c>
3. <https://wiki.ros.org/ROS>
4. Sufiyan Ali, et al. “ROS Based Autonomous Mobile Manipulator Robot.” *Advances in Engineering Research*, vol. 5, no. 1, 2023, pp. 780–789, https://doi.org/10.2991/978-94-6463-252-1_78. Accessed 14 Apr.2024.
5. Vyas, Dhaval R., et al. “Economic 6-DOF Robotic Manipulator Hardware Design for Research and Education.” *Materials Today: Proceedings*, vol. 62, no. 3, 2022, pp. 7179–7184, <https://doi.org/10.1016/j.matpr.2022.03.140>. Accessed 14 Apr. 2024.
6. Priya, Vishwa. “Design and Simulation of Vision Based Robotic Arm Using ROS.” *Volatiles & Essent. Oils*, vol. 8, no. 5, 2021, pp. 3189–3199. Accessed 14 Apr. 2024.

Appendix-A

Installation of ubuntu 20.04 on Virtual box

Installing Ubuntu 20.04 on VirtualBox is a straightforward process. VirtualBox allows you to run Ubuntu as a virtual machine on your host operating system (e.g., Windows, macOS, Linux). Follow these step-by-step instructions to install Ubuntu 20.04 on VirtualBox:

Prerequisites:

1. Download Required Software:

- Download and install Oracle VM VirtualBox from the (<https://www.virtualbox.org/>).
- Download the Ubuntu 20.04 LTS ISO image from the (<https://releases.ubuntu.com/20.04/>).

Steps to Install Ubuntu 20.04 on VirtualBox:

1. Open VirtualBox:

- Launch VirtualBox on your computer.

2. Create a New Virtual Machine:

- Click on "New" to create a new virtual machine.
- Enter a name for your virtual machine (e.g., "Ubuntu 20.04").
- Choose "Linux" as the type and "Ubuntu (64-bit)" as the version.

3. Allocate Memory (RAM):

- Allocate sufficient memory for the virtual machine. A minimum of 2GB is recommended, but more is better for smoother performance.

4. Create a Virtual Hard Disk:

- Select "Create a virtual hard disk now" and click "Create".
- Choose "VDI (VirtualBox Disk Image)" as the hard disk file type.
- Choose "Dynamically allocated" for the storage option (this allows the virtual disk to grow as needed).

5. Allocate Disk Space:

- Choose the amount of disk space you want to allocate for the virtual machine. At least 25GB is recommended for Ubuntu 20.04.

6. Configure Virtual Machine Settings:

- Select the newly created virtual machine from the VirtualBox Manager.
- Go to "Settings" and navigate to the "Storage" section.
- Under "Controller: IDE", click on the empty disk icon and choose the Ubuntu 20.04 ISO image that you downloaded.

7. Start the Virtual Machine:

- Click "Start" to launch the virtual machine.
- The Ubuntu installer should boot from the ISO image.

8. Install Ubuntu 20.04:

- Follow the on-screen instructions to install Ubuntu:
- Choose "Install Ubuntu" from the boot menu.
- Select your language and keyboard layout.
- Choose "Normal installation" and ensure "Download updates while installing Ubuntu" and "Install third-party software" are selected for a more complete installation.
- Choose the installation type (e.g., Erase disk and install Ubuntu).
- Create a user account and set up your system preferences.

9. Complete Installation:

- Once the installation process is complete, restart the virtual machine.
- You may need to "Unmount" the ISO image from the virtual CD/DVD drive in VirtualBox settings to boot into the newly installed Ubuntu.

Post-Installation Steps:**1. Update and Upgrade Ubuntu:**

- Open a terminal in Ubuntu and run the following commands to update the system:

sudo apt update

sudo apt upgrade

2. Install Additional Software:

- Install any additional software or packages you need using the Ubuntu Software Center or command-line tools (`apt`)

Appendix-B

Installation of ROS noetic and Arduino file

a) For ros noetic

Setup ROS Noetic repository

```
sudo sh -c 'echo "deb http://packages.ros.org/ros/ubuntu $(lsb_release -sc) main" >
/etc/apt/sources.list.d/ros-latest.list'
```

```
curl -s https://raw.githubusercontent.com/ros/rosdistro/master/ros.asc | sudo apt-key add -
```

Update packages index and install ROS Noetic Desktop Full

```
sudo apt update
```

```
sudo apt install ros-noetic-desktop-full
```

Initialize rosdep

```
sudo rosdep init
```

```
rosdep update
```

Setup environment

```
echo "source /opt/ros/noetic/setup.bash" >> ~/.bashrc
```

```
source ~/.bashrc
```

Install ROS tools (optional)

```
sudo apt install python3-rosinstall python3-rosinstall-generator python3-wstool build-essential
```

Verify installation

```
rosversion -d
```

```
printenv | grep ROS
```

b) For arduino

Install required packages

```
sudo apt update
```

```
sudo apt install curl git
```

Download and install Arduino CLI

```
curl -fsSL https://raw.githubusercontent.com/arduino/arduino-cli/master/install.sh |
BINDIR=~/.local/bin sh
```

Install Arduino IDE using Arduino CLI

```
arduino-cli core update-index
```

```
arduino-cli core install arduino:avr
```

Install Arduino CLI completion (optional)

```
arduino-cli completion bash > ~/.local/share/bash-completion/completions/arduino-cli
```

Configure udev rules for serial port access

```
sudo usermod -a -G dialout $USER
```

```
sudo usermod -a -G tty $USER
```

Verify Arduino CLI installation

```
arduino-cli version
```

